The Journal of Open Source Software

LikelihoodProfiler.jl: Unified profile-likelihood workflows for identifiability and confidence intervals

Ivan Borisov [1], **Aleksander Demin** [2], **and Evgeny Metelkin** [1]

1 InSysBio CY LTD, Limassol, Cyprus **2** National Research University Higher School of Economics, Moscow, Russia

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Owen Lockwood

Reviewers:

- @salbalkus
- @currocam
- @dufourc1

Submitted: 04 September 2025
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

Practical identifiability addresses how well a mechanistic model is determined by available experimental data. Profile likelihood-based methods are widely used to explore practical identifiability and often serve as a proxy for structural identifiability analysis (Heinrich et al., 2025). Moreover, these methods can also be generalized to states and predictions analysis. This versatility makes profile likelihood an essential component in model development.

The paper presents `LikelihoodProfiler.jl`, an open-source Julia package for conducting profile likelihood-based identifiability and predictability analysis.

Statement of Need

Current profile likelihood implementations are often tied to specific modeling frameworks or software environments. In addition, different profile likelihood algorithms are spread across separate tools, forcing users to switch between languages and interfaces. `LikelihoodProfiler.jl` addresses these limitations by providing: - Unified interface to multiple profile likelihood algorithms. - Compatibility with common modeling standards (Heta (Metelkin, 2021), SBML, PEtab (Persson & others, 2025)). - Integration with Julia's SciML (Rackauckas & Nie, 2017) for efficient computation and extensibility.

Features and Methodologies

`LikelihoodProfiler.jl` provides a unified interface for various profile likelihood methods, including optimization-based **OptimizationProfiler** and integration-based profiles **IntegrationProfiler**, CI endpoints search **CICOProfiler**, and more.

All methods leverage a `CommonSolve` interface (Rackauckas & Nie, 2017) (`CommonSolve.solve()`), supporting global settings for parallelization and verbosity. Profiling results are directly visualizable using the `Plots.jl` package and exportable as DataFrames for further analysis.

Demonstrative Example: JAK/STAT Signaling Pathway Model

`LikelihoodProfiler.jl`'s functionality and interfaces are demonstrated using the JAK/STAT signaling pathway model (Boehm et al., 2014), which consists of 8 states and 9 parameters. The model and experimental data were sourced from the Benchmark-Models-PEtab repository (contributors, 2024) and imported through the `PEtab.jl` package (Persson & others, 2025).

```julia
using LikelihoodProfiler, CICOBase, PEtab, Plots
petab_model = PEtabModel("Boehm_JProteomeRes2014.yaml")
petab_problem = PEtabODEProblem(petab_model)
```

To study the identifiability of the JAK/STAT model parameters, we first construct a `ProfileLikelihoodProblem` and select an appropriate profiling method. A

`ProfileLikelihoodProblem` is defined by providing (i) the objective function (typically the negative log-likelihood) and (ii) the initial parameter values corresponding to the optimum of this objective. `LikelihoodProfiler.jl` builds on the `Optimization.jl` interface (Dixit & Rackauckas, 2023), and `ProfileLikelihoodProblem` wraps an `OptimizationProblem` defined in `Optimization.jl`. In addition, `ProfileLikelihoodProblem` allows users to specify optional arguments common across profiling methods, such as the indices of parameters to profile, upper and lower bounds, and other options.

```
using Optimization, OrdinaryDiffEq
optprob = OptimizationProblem(petab_problem)
param_profile_prob = ProfileLikelihoodProblem(optprob, get_x(petab_problem); idxs=1:3)
```

`LikelihoodProfiler.jl` offers a suite of methods for solving the `ProfileLikelihoodProblem`. In this example, we use the Hessian-free variant of the `IntegrationProfiler` (Chen & Jennrich, 2002), which approximates the likelihood profile and performs re-optimization after each ODE solver step to prevent divergence from the true profile trajectory. Each profiling method offers several configurable options.

```
alg = IntegrationProfiler(integrator = Tsit5(), integrator_opts = (dtmax=0.5, reltol=1e-
  matrix_type = :identity, gamma=0., reoptimize=true,
  optimizer = Optimization.LBFGS(),optimizer_opts=(maxiters=10000,))

sol_param = solve(param_profile_prob, alg, verbose=true)
```

An alternative approach, implemented in `CICOProfiler`, estimates the confidence intervals (CI) endpoints directly — without reconstructing the full profile — by solving a constrained optimization problem (Borisov & Metelkin, 2020). This method is often more efficient when only the CI is required.

```
alg_cico = CICOProfiler(optimizer = :LN_NELDERMEAD, scan_tol = 1e-10)
```

Users can get estimated confidence intervals using the `endpoints(sol_param)` function or visualize profile curves with `plot()`.

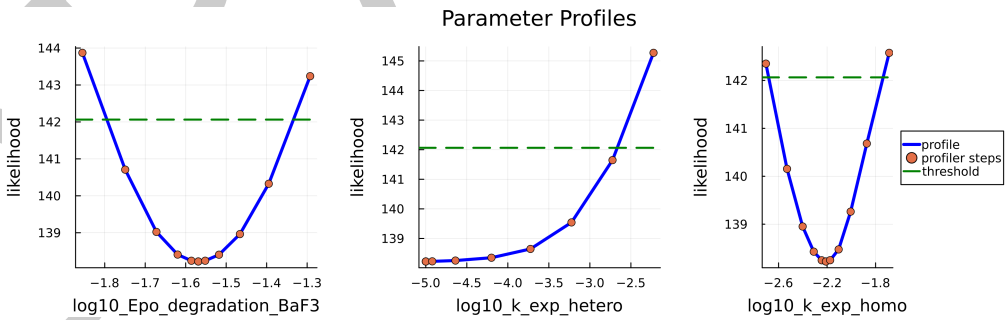

Figure 1: Parameter profiles

DRAFT

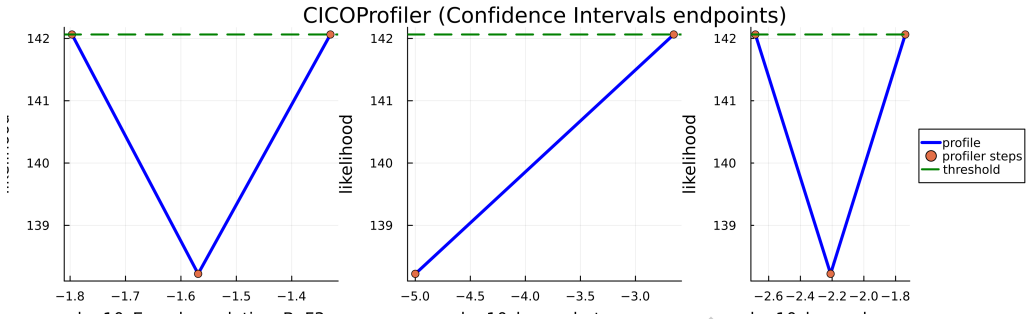

Figure 2: CICO profiles

The same algorithms can also be applied to arbitrary functions of parameters. This generalization of profile likelihood concept (Kreutz et al., 2012) is used to study model reparametrization or perform predictability analysis. This functionality is available within the same interface: users define functions of parameters and provide them as the third argument to the problem. As an illustration, we define a function that returns the observable value at selected time points:

```
function pSTAT5A_rel_obs(x, p, t)
  sol = get_odesol(x, petab_problem)(t)
  specC17 = 0.107
  return (100 * sol[7] + 200 * sol[6] * specC17) / (sol[7] + sol[1] * specC17 + 2 * sol[
end
pSTAT5A_rel_optf(t) = OptimizationFunction((x, p) -> pSTAT5A_rel_obs(x, p, t), AutoFinit

times = [2.5, 60.0, 200.0]
func_profile_prob = ProfileLikelihoodProblem(optprob, get_x(petab_problem), [pSTAT5A_rel
  profile_lower=0.0, profile_upper=120.0)

sol_func = solve(func_profile_prob, alg)
```

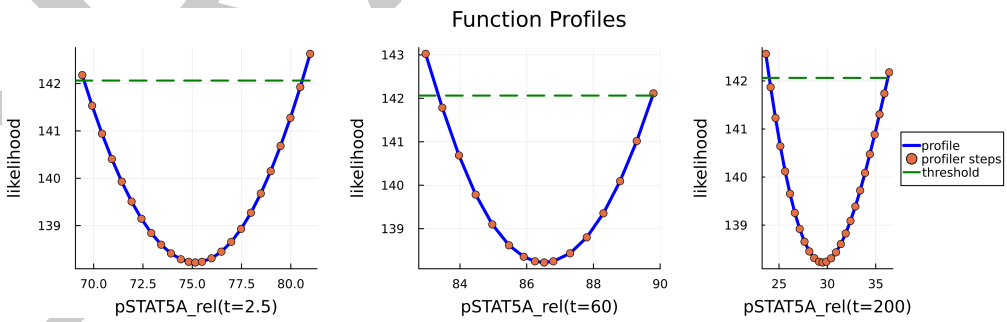

Figure 3: Function profiles

Prediction confidence interval endpoints can be estimated at a larger set of time points, and smooth prediction bands can then be plotted.

DRAFT

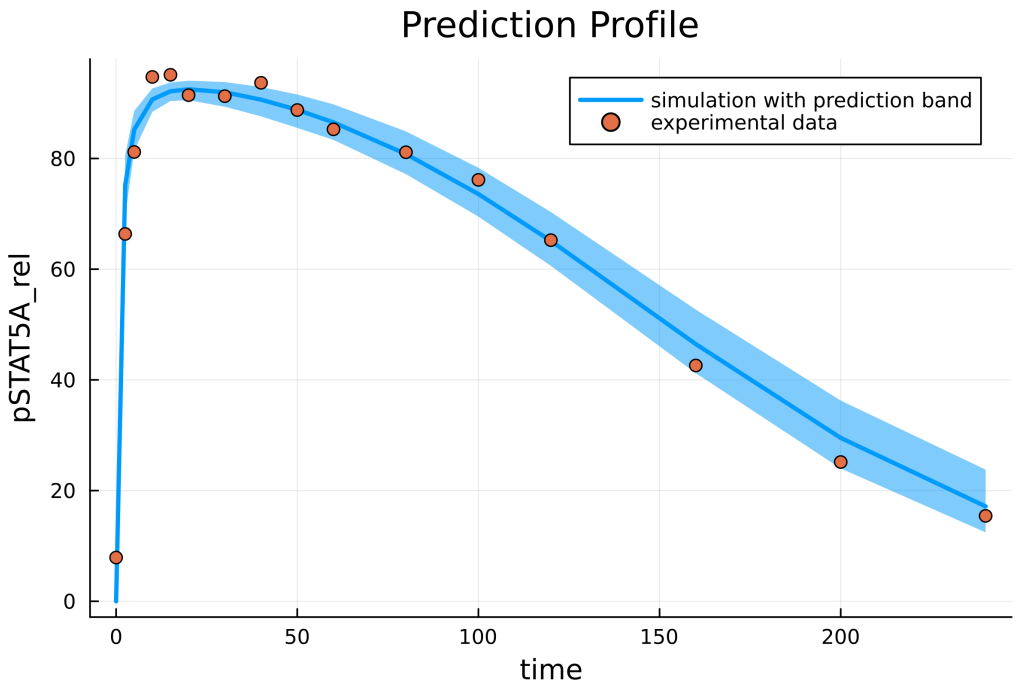

Figure 4: Prediction profile

Implementation and Extensibility

All profiling methods benefit from the unified interface provided by `LikelihoodProfiler.jl`: - Integration with `SciML` packages gives users access to a wide range of optimizers, differential equation solvers, and AD backends, enabling efficient profiling configurations. - Compatibility with `Heta`, `PEtab` and `SBML` formats broadens the accessibility of the package across different modeling frameworks. - `solve()` interface provided by `CommonSolve.jl` provides unified access various profiling methods which can be used both for parameters and for functions identifiability analysis. - A common parallelization setup, controlled via the `parallel_type` argument in the `solve()` function, is supported across all methods and can significantly accelerate computations. - The interface facilitates integration of new profiling methods and stepping algorithms.

Future work will include adding new methods of parameters, functions and predictions profiling and enabling adaptive switching between strategies.

Availability

LikelihoodProfiler.jl is open-source and available at: https://github.com/insysbio/LikelihoodProfiler.jl

The package is registered in Julia and can be installed from the Julia REPL using:

```
import Pkg; Pkg.add("LikelihoodProfiler")
```

Tutorials and documentation are available at: https://insysbio.github.io/LikelihoodProfiler.jl/stable/

Related packages

Julia packages, such as `ProfileLikelihood.jl` and `InformationGeometry.jl`, offer related functionality but do not provide multiple profiling methods through a unified interface. Outside Julia, tools like `Data2Dynamics (MATLAB)`, `dMod (R)`, and `pyPESTO (Python)` combine

DRAFT

parameter estimation and profile likelihoods. However, these frameworks are typically tied to their respective modeling environments and are not designed to support identifiability analysis independently of the model-development workflow.

References

Boehm, M. E., Adlung, L., Schilling, M., Roth, S., Klingmüller, U., & Lehmann, W. D. (2014). Identification of isoform-specific dynamics in phosphorylation-dependent STAT5 dimerization by quantitative mass spectrometry and mathematical modeling. *J. Proteome Res.*, *13*(12), 5685–5694. https://doi.org/10.1021/pr5006923

Borisov, I., & Metelkin, E. (2020). Confidence intervals by constrained optimization—an algorithm and software package for practical identifiability analysis in systems biology. *PLoS Comput Biol*, *16*(12), e1008495. https://doi.org/10.1371/journal.pcbi.1008495

Chen, J.-S., & Jennrich, R. I. (2002). Simple accurate approximation of likelihood profiles. *Journal of Computational and Graphical Statistics*, *11*(3), 714–732. https://doi.org/10.1198/106186002493

contributors, T. Pe. B. C. (2024). *The PEtab Benchmark Collection of parameter estimation problems* (Version 2024.10.15). Zenodo. https://doi.org/10.5281/zenodo.8155057

Dixit, V. K., & Rackauckas, C. (2023). *Optimization.jl: A unified optimization package* (Version v3.12.1). Zenodo. https://doi.org/10.5281/zenodo.7738525

Heinrich, M., Rosenblatt, M., Wieland, F.-G., Stigter, H., & Timmer, J. (2025). On structural and practical identifiability: Current status and update of results. *Current Opinion in Systems Biology*, *50*, 100546. https://doi.org/10.1016/j.coisb.2025.100546

Kreutz, C., Raue, A., & Timmer, J. (2012). Likelihood based observability analysis and confidence intervals for predictions of dynamic models. *BMC Systems Biology*, *6*(120), 1–12. https://doi.org/10.1186/1752-0509-6-120

Metelkin, E. (2021). Heta compiler: A software tool for the development of large-scale QSP models and compilation into simulation formats. *JOSS*, *6*(67), 3708. https://doi.org/10.21105/joss.03708

Persson, S., & others. (2025). *PEtab.jl: Advancing the efficiency and utility of dynamic modelling*. https://doi.org/10.1101/2025.04.30.651378

Rackauckas, C., & Nie, Q. (2017). DifferentialEquations.jl – a performant and feature-rich ecosystem for solving differential equations in julia. *JORS*, *5*(1), 15. https://doi.org/10.5334/jors.151

DRAFT